

Unit-6: Backtracking

- In the search for fundamental principles of algorithm design, backtracking represents one of the most general techniques. Many problems which deal with searching for a set of solutions or which ask for an optimal solution satisfying some constraints can be solved using the backtracking formulation.
- A backtracking algorithm is a problem-solving strategy that uses a brute force approach for finding the desired output. The Brute force approach tries out all the possible solutions and chooses the desired/best solutions.
- The term backtracking suggests that if the current solution is not suitable, then backtrack and try other solutions
- It uses recursive calling to find a solution set by building a solution step by step, increasing levels with time. In order to find these solutions, a search tree named state-space tree is used. In a state-space tree, each branch is a variable, and each level represents a solution.
- Backtracking is an important tool for solving constraint satisfaction problems.
- Approach of finding solution(s) by trying all possible paths and then abandoning them if they are not suitable.
 - A "brute force" algorithmic technique (tries all paths)
 - Often implemented recursively
 - Could involve looking for one solution
 - If any of the paths found a solution, a solution exists! If none find a solution, no solution exists
 - Could involve finding all solutions
 - It's exhaustive search (Exploring every possible combination from a set of choices or values.) with conditions
 - DFS strategy is used

In order to apply the backtrack method, the desired solution must be expressible as an n -tuple $(x_1, \dots, x_n)$ where the x_i are chosen from some finite set S_i . Often the problem to be solved calls for finding one vector which maximizes (or minimizes or satisfies) a criterion function $P(x_1, \dots, x_n)$. Sometimes it seeks all such vectors which satisfy P .

- Many of the problems we shall solve using backtracking require that all the solutions satisfy a complex set of **constraints**. For any problem these **constraints may be divided into two categories: explicit and implicit**. **Explicit constraints** are rules which restrict each x_i to take on values only from a given set. Common examples of explicit constraints are:

$$\begin{array}{ll} x_i \geq 0 & \text{or } S_i = \{\text{all nonnegative real numbers}\} \\ x_i = 0 \text{ or } 1 & \text{or } S_i = \{0, 1\} \\ l_i \leq x_i \leq u_i & \text{or } S_i = \{a: l_i \leq a \leq u_i\} \end{array}$$

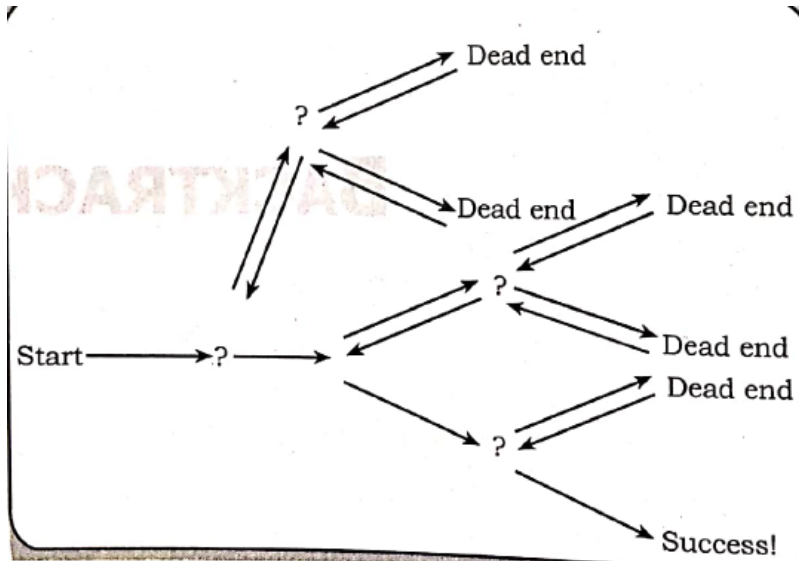
The explicit constraints may or not depend on the particular instance I of the problem being solved. All tuples that satisfy the explicit constraints define a possible solution space for I . **The implicit constraints** determine which of the tuples in the solution space of I actually satisfy the criterion function. Thus implicit constraints describe the way in which the x_i must relate to each other.

Example: For 8-queens problem:

Explicit constraints using 8-tuple formation, for this problem are $S = \{1, 2, 3, 4, 5, 6, 7, 8\}$.

The implicit constraints for this problem are that no two queens can be the same (i.e., all queens must be on different columns) and no two queens can be on the same diagonal.

- A **backtracking algorithm uses the depth-first search** method. When it starts exploring the solutions, a bounding function is applied so that the algorithm can check if the so-far built solution satisfies the constraints. If it does, it continues searching. If it doesn't, the branch would be eliminated, and the algorithm goes back to the level before.



Example Backtracking Approach

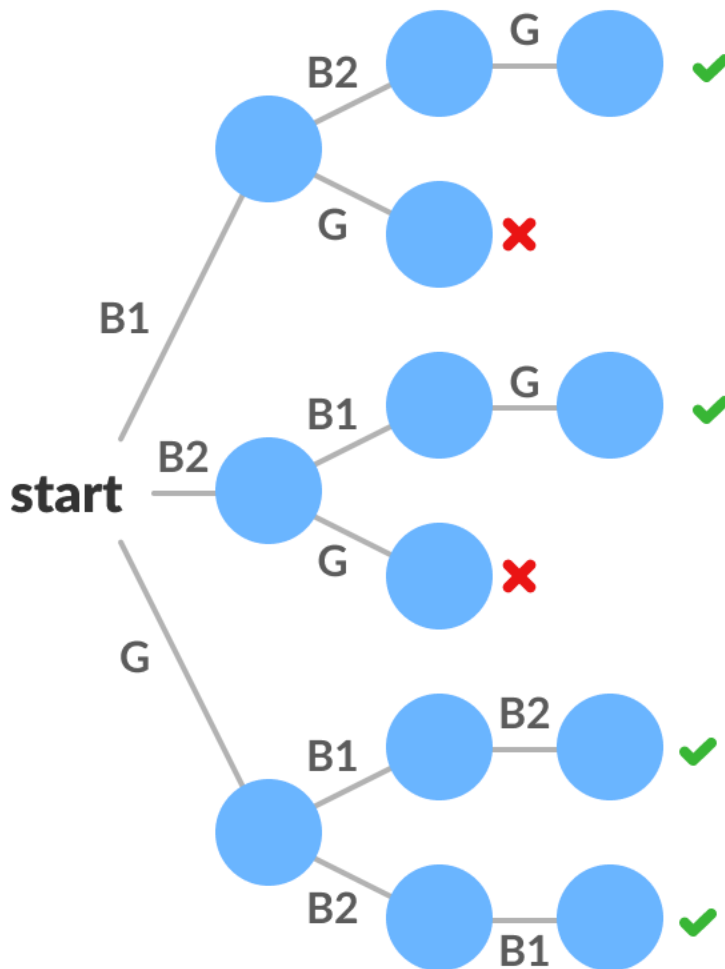
Problem: You want to find all the possible ways of arranging 2 boys and 1 girl on 3 benches.
Constraint: Girl should not be on the middle bench.

Solution: There are a total of $3! = 6$ possibilities. We will try all the possibilities and get the possible solutions. We recursively try all the possibilities.

All the possibilities are:



The following state space tree shows the possible solutions.



Recursion vs backtracking

In recursion, the function calls itself until it reaches a base case. In backtracking, we use recursion to explore all the possibilities until we get the best result for the problem. In recursion, no values are abandoned but in backtracking some solution candidates can be abandoned.

Applications of backtracking approach

- To solve N Queen Problem
- To find all Hamiltonian Cycles present in a graph.
- To solve graph coloring problem
- To solve Sub-set sum problem
- Solving 0-1 Knapsack problem

Sub-set sum problem

- Given a set of positive integers and a sum value. We need to find all possible subsets of the set with a sum equal to the sum value.
- Example:
Set: S = {10, 7, 5, 18, 12, 20, 15} Sum: 30
Output: [{10, 5, 15}, {10, 20}, {7, 5, 18}, {18, 12}]
- The backtracking approach generates all permutations in the worst case but in general, performs better than the recursive approach
- The problem can be solved by using dynamic programming approach.

Backtracking algorithm for subset sum problem

Input: Sorted (in non-decreasing order) array, w of n positive integers and another positive integer m .

Output: All possible combinations of elements of w, whose sum is m

```

SubSetSum(i, sumSofar, sumLeft){
    if(promising(i, sumSofar, sumLeft)){// may lead to solution
        if(sumSofar == m ){
            print solution;
        }
        else{ // expand the node when sumSofar < m
            include[i+1] = 1; //include w[i+1]
            SubSetSum(i+1, sumSofar+w[i+1], sumLeft-w[i+1]);
            include[i+1] = 0; // do not include w[i+1]
            SubSetSum(i+1, sumSofar, sumLeft-w[i+1]);
        }
    }
}

promising(i, sumSofar, sumLeft) {
    return (sumSofar+sumLeft >= m && (sumSofar == m || sumSofar + w[i+1] <= m))
}

```

Note: Initial call : SubSetSum (-1, 0, totalSumOfElements)

Analysis:

Time complexity : In worst case (when all possible subsets are generated) the algorithm requires $O(2^n)$

Example: $w = \{3, 5, 6, 7\}$ and $m = 15$

The following pruned state space tree only have 15 nodes. The full state space tree has $(2^{4+1} - 1 = 31)$ nodes. Notice that $w[0]$ is included in level 1 of the tree, $w[1]$ is included in level 2 of the tree and so on. Root of the tree contains $\text{sumSofar} = 0$ and $\text{sumLeft} = \text{totalSumOfElements}$.

- A node is **expanded** if it is **promising node** i.e

$\text{if}(\text{sumSofar} + \text{sumLeft} \geq m \ \&\& \ (\text{sumSofar} == m \ || \ \text{sumSofar} + w[i+1] \leq m))$

and included in the subset.

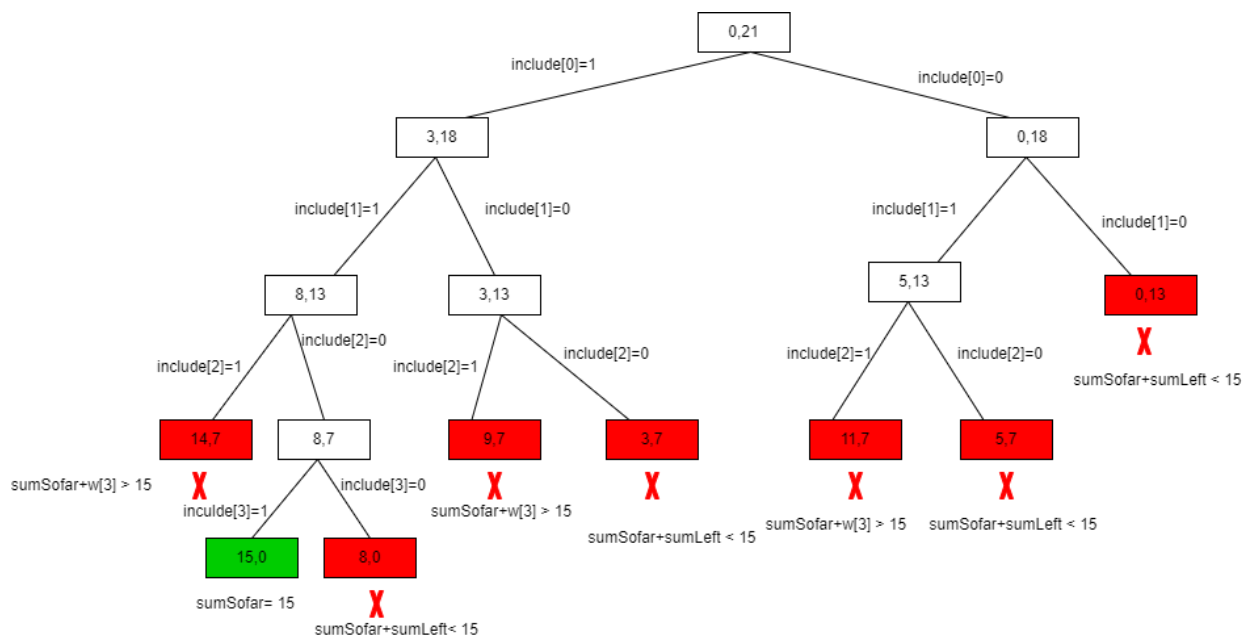
Otherwise the **node is not expanded and backtracked** to appropriate node.

[Note: initially $i = -1$]

⇒ A node is represented as

sumSofar, sumLeft

⇒ Cross symbol means can not be expanded further and needs to be backtracked



The process can be explained from following table where include is an array whose values indicate which items of $w = \{3,5,6,7\}$ are included in the solution subset and which items are not.

i	node	sumSofar	sumLeft	Comment	include[]
-1	1	0	21	Start	{0,0,0,0 }
0	2	3	18	Include 3	{1,0,0,0}
1	3	8	13	Include 5	{1,1,0,0}
2	4	14	7	Include 6	{1,1,1,0}
(since sumSofar (14) + next item (7) > 15 (m value), backtrack)					
2	5	8	7	Exclude 6	{1,1,0,0}
3	6	15	0	Include 7	{1,1,0,1}
(Since sumSofar = 15 = m, Print solution {3,5,7}, backtrack)					
3	7	8	0	Exclude 7	{1,1,0,0}
(Since sumSofar(8) + sumLeft(0) < m(15) , backtrack)					
1	8	3	13	Exclude 5	{1,0,0,0}
2	9	9	7	Include 6	{1,0,1,0}
(since sumSofar (9) + next item (7) > 15 (m value), backtrack)					
2	10	3	7	Exclude 6	{1,0,0,0}
(since sumSofar (3) + sumLeft (7) < 15 (m value), backtrack					
0	11	0	18	Exclude 3	{0,0,0,0}
1	12	5	13	Include 5	{0,1,0,0}
2	13	11	7	Include 6	{0,1,1,0}
(since sumSofar (11) + next item (7) > 15 (m value), backtrack)					
2	14	5	7	Exclude 6	{0,1,0,0}
(since sumSofar(5) + sumLeft(7) < m(15) , backtrack					
1		0	13	Exclude 5	{0,0,0,0}
(since sumSofar(0) + sumLeft(13) < m(15) , stop					

Exercise: Trace sub set sub algorithm for $w = \{5, 10, 12, 13, 15, 18\}$ and $m = 30$

Backtracking approach for 0-1 Knapsack Problem

Problem: Given n items each having a weight and a profit (value) and the capacity of knapsack W . Determine the set of items with maximum total profit under the constraint that the sum of their weights cannot exceed W .

Recall that in 0-1 knapsack problem, an item can be either selected in whole or not selected. (Fraction of item is not allowed)

Points to considered while solving the problem using backtracking

- In stat space tree a node is represented as

profit, weight, bound

- If the node is at level i , and the node at level k is the one that would bring the sum of the weights above W . Then

$$totweight = weight + \sum_{j=i+1}^{k-1} w_j, \quad \text{and}$$

$$bound = \underbrace{\left(profit + \sum_{j=i+1}^{k-1} p_j \right)}_{\text{Profit from first } k-1 \text{ items taken}} + \underbrace{(W - totweight)}_{\text{Capacity available for } k\text{th item}} \times \underbrace{\frac{p_k}{w_k}}_{\text{Profit per unit weight for } k\text{th item}}$$

Where,

profit = the sum of the profits of the items included up to the node

weight = the sum of the weights of those items included up to the node

W = capacity of the knapsack

- Conditions for backtracking (the node is non-promising and should not be expanded):
 - $weight \geq W$. OR $bound \leq maxprofit$.

Where,

$maxprofit$ = the value of the profit in the best solution found so far.

(Initially $maxprofit = 0$)

NOTE:

- We initialize variables *bound* and *totweight* to *profit* and *weight*, respectively. Next we greedily grab items, adding their profits to *bound* and their weights to *totweight*, until we get to an item that if grabbed would bring *totweight* above *W*.
- We grab the fraction of that item allowed by the remaining weight, and we add the value of that fraction to *bound*. If we are able to get only a fraction of this last weight, this node cannot lead to a profit equal to *bound*, but *bound* is still an upper bound on the profit we could achieve by expanding beyond the node.
- We are using greedy considerations only to obtain a bound that tells us whether we should expand beyond a node. We are not using it to greedily grab items with no opportunity to reconsider later (as is done in the greedy approach).

Example: Following table represents the item with their weights and profits and are ordered in non-increasing order, according to the values of $p[i]/w[i]$.

i	1	2	3	4
p _i (\$)	40	30	50	10
w _i (Kg)	2	5	10	5

Initially,

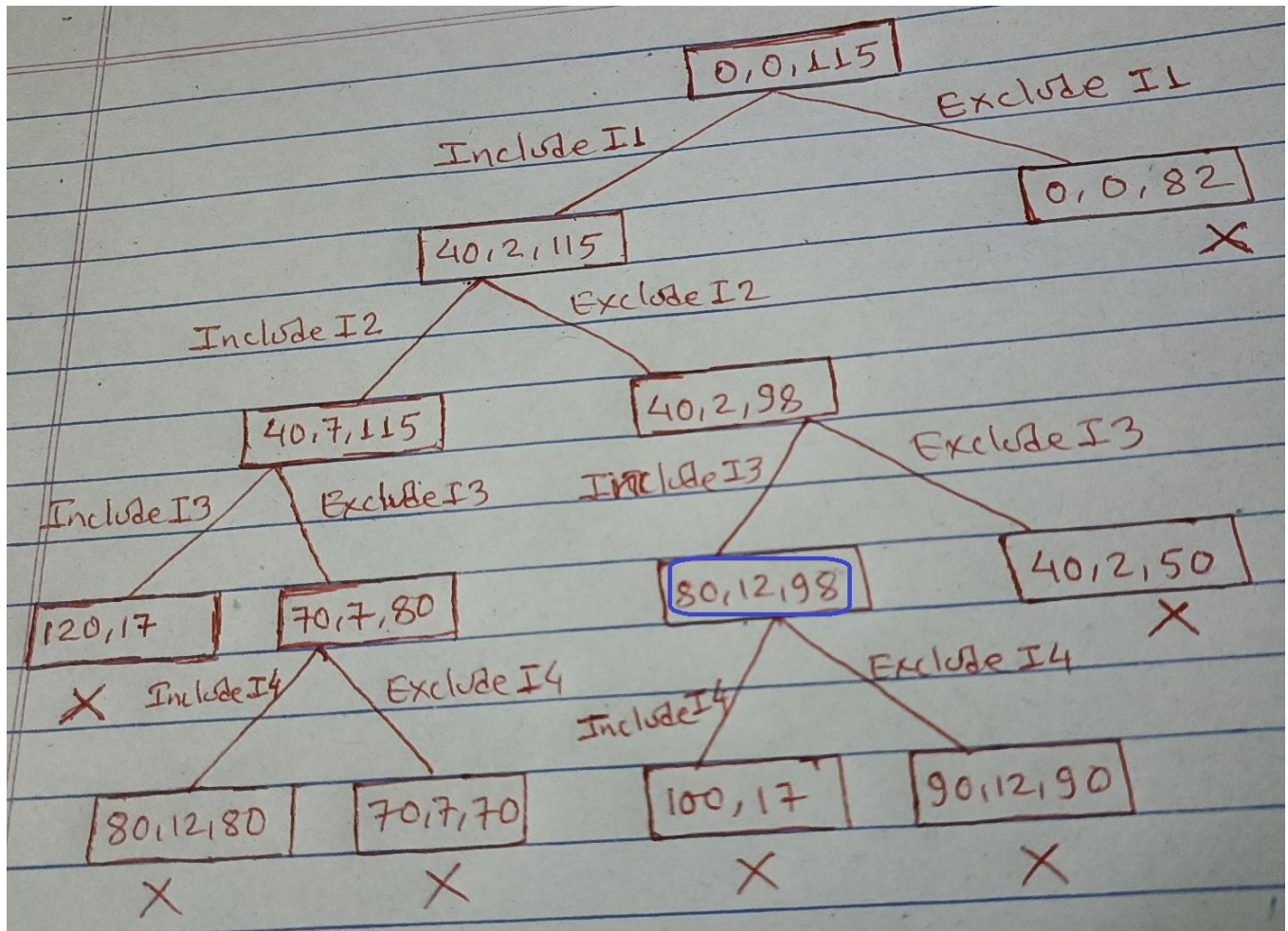
$$profit = \$0$$

$$weight = 0$$

Because $2 + 5 + 10 = 17$, and $17 > 16$, the value of *W*, the third item would bring the sum of the weights above *W*. Therefore, $k = 3$, and we have

$$\begin{aligned}
 totweight &= weight + \sum_{j=0+1}^{3-1} w_j = 0 + 2 + 5 = 7 \\
 bound &= profit + \sum_{j=0+1}^{3-1} p_j + (W - totweight) \times \frac{p_3}{w_3} \\
 &= \$0 + \$40 + \$30 + (16 - 7) \times \frac{\$50}{10} = \$115.
 \end{aligned}$$

(IDEA: bound means, what would be total profit if you can grab all remaining items filling the knapsack (even by taking the fraction) beyond that node (item))



See in level 3, leftmost node **120,17** we should backtrack here since weight is greater than capacity of knapsack (bound cannot be computed in this situation) .

See in level 3, rightmost node **40,12,50** we should backtrack here since maxprofit at this stage is 80 (not 40!) which was set before and the bound value is 50 (since bound < maxprofit). Same concept in case of level1 rightmost node.

Here, only a possible solution is taking Item1 and Item three with maximum profit of 80 and total weight taken is 12. See the marked node 80,12,98

Refer book , “*Foundations of Algorithms*”.by Richard E. Neapolitan for detail

Algorithm

Problem: Let n items be given, where each item has a weight and a profit. The weights and profits are positive integers. Furthermore, let a positive integer W be given. Determine a set of items with maximum total profit, under the constraint that the sum of their weights cannot exceed W .

Inputs: Positive integers n and W ; arrays w and p , each indexed from 1 to n , and each containing positive integers sorted in non-increasing order according to the values of $p[i] / w[i]$.

Outputs: an array *bestset* indexed from 1 to n , where the values of *bestset*[i] is “yes” if the i th item is included in the optimal set and is “no” otherwise; an integer maxprofit that is the maximum profit.

```

void knapsack (index i,
              int profit, int weight)
{
    if (weight <= W && profit > maxprofit){
        maxprofit = profit;
        numbest = i;
        bestset = include;
    }

    if (promising(i)){
        include[i + 1] = "yes";
        knapsack(i + 1, profit + p[i + 1], weight + w[i + 1]);
        include[i + 1] = "no";
        knapsack(i + 1, profit, weight);
    }
}

bool promising (index i)
{
    index j, k;
    int totweight;
    float bound;

    if (weight >= W)
        return false;
    else{
        j = i + 1;
        bound = profit;
        totweight = weight;
        while (j <= n && totweight + w[j] <= W){
            totweight = totweight + w[j];
            bound = bound + p[j];
            j++;
        }
        k = j;
        if (k <= n)
            bound = bound + (W - totweight) * p[k] / w[k];
        return bound > maxprofit;
    }
}

```

NOTE: n , w , p , W , $maxprofit$, $include$, $bestset$, and $numbest$ are not inputs to either routine. If these variables were defined globally, the following code would produce the maximum profit and a set that has that profit:

```

numbest = 0;
maxprofit = 0;
knapsack(0, 0, 0);
cout << maxprofit;           // Write the maximum profit.
for (j = 1; j <= numbest; j++) // Show an optimal set of items.
    cout << bestset[i];

```

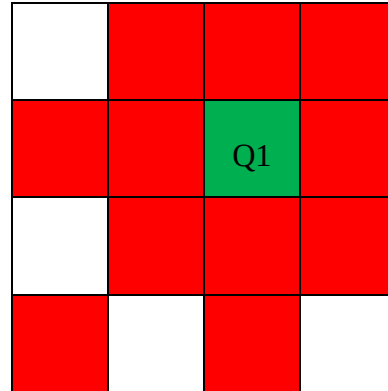
Analysis

The backtracking algorithm to solve 0-1 requires $O(2^n)$ in worst case since 2^n nodes are needed to be checked to decide either to include or exclude.

N-Queen Problem

Problem: We are given N chess's queens and asked to place them in the chessboard of size $N \times N$ so that no two queens attack/eliminate each other. A queen can only attack if another queen is either on:

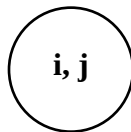
- Same Row
- Same Column
- Same Diagonal (left or right)



(If Queen Q1 is placed as in above figure next queen cannot be placed in the marked places.)

Points to considered while solving the problem using backtracking

⇒ A node is represented as



Meaning: the queen in i^{th} row is placed at j^{th} column

Condition for backtracking (i.e. node is non-promising)

- Queen at row i and Queen at row k are in same column i.e.

$$col(i) = col(k).$$

Here, $col(i)$ is the column where the queen in the i^{th} row is located, and $col(k)$ is the column where the queen in the k^{th} row is located

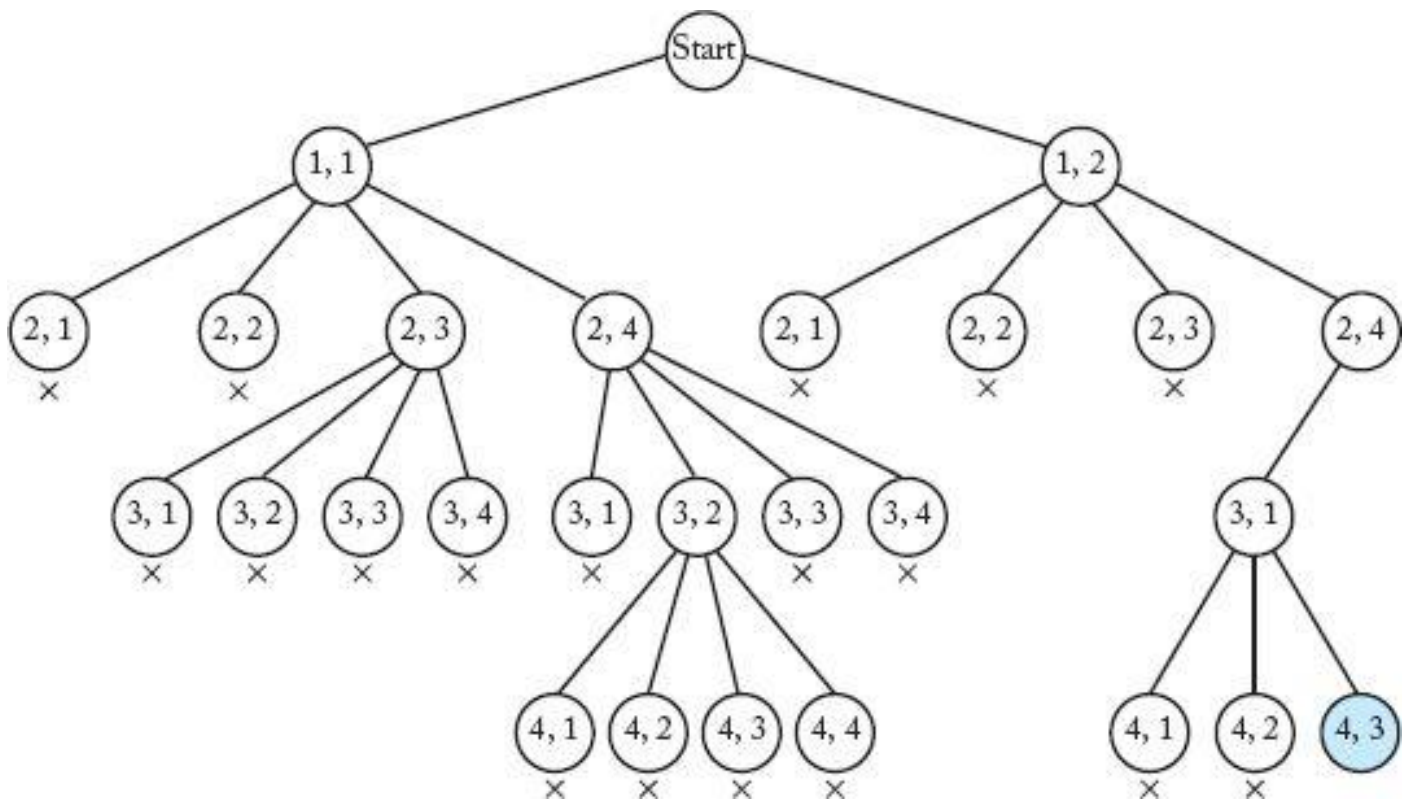
- The queen in the k^{th} row threatens the queen in the i^{th} row along one of its diagonals i.e.

$$col(i) - col(k) = i - k \quad \text{or} \quad col(i) - col(k) = k - i.$$

- ✎ For the queen threatening from the left diagonal, the difference in the columns is the same as the difference in the rows.
- ✎ Similarly, for the queen threatening from the right diagonal, the difference in the columns is the negative of the difference in the rows.

Example: Pruned state space tree for **4-Queen problem**

- ✎ A portion of the pruned state space tree produced when backtracking is used to solve the instance of the n-Queens problems in which $n = 4$ is given below.
- ✎ Only the nodes checked to find the first solution are shown. That solution is found at the color-shaded node. Each nonpromising node is marked with a cross.

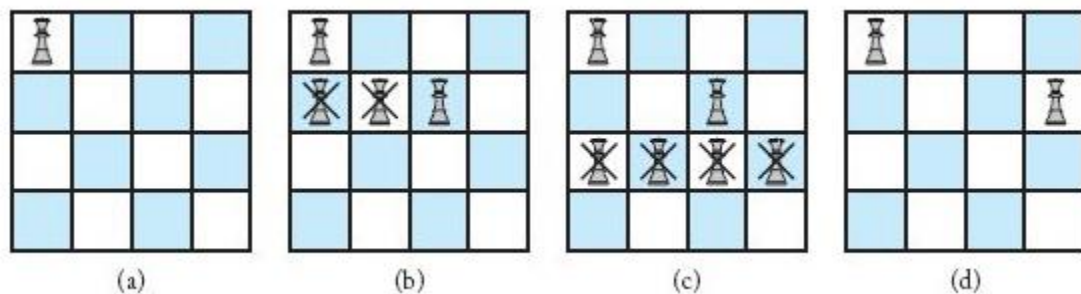


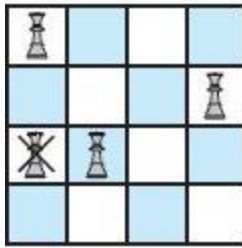
- (a) $\langle 1, 1 \rangle$ is promising. {because queen 1 is the first queen positioned}
- (b) $\langle 2, 1 \rangle$ is nonpromising. {because queen 1 is in column 1}
 $\langle 2, 2 \rangle$ is nonpromising. {because queen 1 is on left diagonal}
 $\langle 2, 3 \rangle$ is promising.
- (c) $\langle 3, 1 \rangle$ is nonpromising. {because queen 1 is in column 1}
 $\langle 3, 2 \rangle$ is nonpromising. {because queen 2 is on right diagonal}
 $\langle 3, 3 \rangle$ is nonpromising. {because queen 2 is in column 3}
 $\langle 3, 4 \rangle$ is nonpromising. {because queen 2 is on left diagonal}
- (d) Backtrack to $\langle 1, 1 \rangle$.
 $\langle 2, 4 \rangle$ is promising.
- (e) $\langle 3, 1 \rangle$ is nonpromising. {because queen 1 is in column 1}
 $\langle 3, 2 \rangle$ is promising. {This is the second time we've tried $\langle 3, 2 \rangle$.}

TASK: Please complete the tree and get all possible solutions

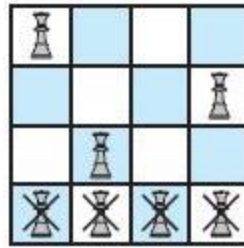
- ✎ The actual chessboard positions that are tried when backtracking is used to solve the instance of the n-Queens problem in which $n = 4$ are represented in following figures. Each nonpromising position is marked with a cross.

(The following figures illustrates the steps to get one of the possible solutions. Please continue the steps to get other solutions)

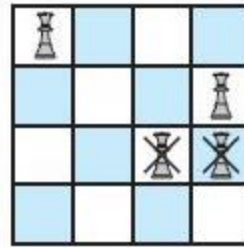




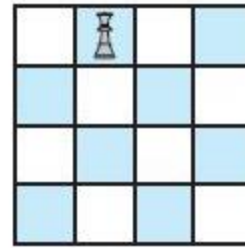
(e)



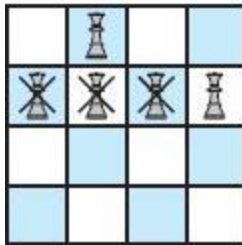
(f)



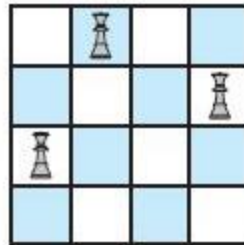
(g)



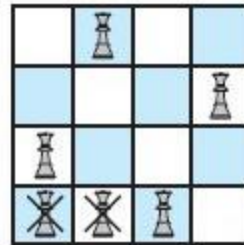
(h)



(i)



(j)



(k)

The Backtracking Algorithm for the n-Queens Problem

Problem: Position n queens on a chessboard so that no two are in the same row, column, or diagonal.

Inputs: positive integer n .

Outputs: all possible ways n queens can be placed on an $n \times n$ chessboard so that no two queens threaten each other. Each output consists of an array of integers col indexed from 1 to n , where $col[i]$ is the column where the queen in the i th row is placed.

```
void queens (index i)
{
    index j;

    if (promising(i))
        if (i == n)
            cout << col[1] through col [n];
        else
            for (j = 1; j <= n; j++){           // See if queen in
                col[i + 1] = j;                 // (i + 1)st row can be
                queens(i + 1);                  // positioned in each of
                                                // the n columns.
            }
}

bool promising (index i)
{
    index k;
    bool switch;

    k = 1;
    switch = true;           // Check if any queen threatens
    while (k < i && switch){ // queen in the ith row.
        if (col[i] == col[k] || abs(col[i] - col[k]) == i - k)
            switch = false;
        k++;
    }
    return switch;
}
```

Note: If the algorithm were implemented by defining n and col globally, the top-level call to *queens* would be

```
queens(0);
```

Analysis

- ✎ We can get an upper bound on the number of nodes in the pruned state space tree by counting the number of nodes in the entire state space tree. This latter tree contains 1 node at level 0, n nodes at level 1, n^2 nodes at level 2, ... , and n^n nodes at level n . The total number of nodes is

$$1 + n + n^2 + n^3 + \dots + n^n = \frac{n^{n+1} - 1}{n - 1}.$$

- ✎ Hence Time complexity is $O(n^n)$
- ✎ Solution of n Queen problem using backtracking checks for all possible arrangements of n Queens on the chessboard. And then checks for validity of the solution. The number of possible arrangements of n Queen on $n \times n$ chessboard is $n!$. So average and worst case complexity of this solution is $O(n!)$. The best case occurs if we find our solution before expanding all possible arrangements. This depends on our implementation. And if we need all possible solutions, the best, average and worst case complexity remains $O(n!)$